

GUIDA DEFINITIVA

LVGL + SquareLine Studio su ESP32-S3-Touch- AMOLED-1.91

Configurazione e sviluppo in Arduino IDE · Waveshare 1.91" AMOLED 536×240 ·
SH8601 + FT3168

Dalla struttura del progetto al modello a task + mutex, dagli eventi SquareLine al comando dei GPIO e del pannello, fino a WiFi, SD, sensori e tuning di memoria.

1. A cosa serve questa guida

Questo documento raccoglie tutto il necessario per partire da uno sketch Arduino "pulito" e costruirci sopra un'interfaccia LVGL disegnata in SquareLine Studio, sfruttando tutto ciò che la scheda offre: display AMOLED, touch capacitivo, WiFi/BLE, microSD, IMU. L'impostazione è quella di uno **starter riutilizzabile**: il boilerplate di basso livello sta in un modulo dedicato, e nello sketch resta solo la tua logica.

Correzione importante rispetto alle prime versioni dello starter. Sulla 1.91 la microSD **condivide fisicamente i pin col display** (clock su GPIO47, dato su GPIO8). Non è quindi possibile metterla "su un bus SPI completamente separato": vedi la sezione SD per l'approccio corretto.

2. La scheda in breve

La WaveShare ESP32-S3-Touch-AMOLED-1.91 monta un **ESP32-S3** dual-core LX7 fino a 240 MHz, con **16 MB di Flash** e **8 MB di PSRAM** (octal). Il display è un **AMOLED da 1.91"**, risoluzione **536×240**, controller **SH8601** su interfaccia **QSPI**. Il touch (versione Touch) usa il controller **FT3168** capacitivo a 5 punti su **I2C**. A bordo ci sono anche un IMU 6 assi **QMI8658**, un RTC, lo slot **microSD** (TF) e l'header per batteria al litio.

SoC	ESP32-S3, dual-core Xtensa LX7, fino a 240 MHz, WiFi 2.4 GHz + BLE 5
Memoria	512 KB SRAM interna, 16 MB Flash, 8 MB PSRAM octal (modulo N16R8)
Display	AMOLED 1.91", 536×240, 65K colori, controller SH8601, bus QSPI
Touch	FT3168 capacitivo, I2C, 5 punti, indirizzo 0x38
IMU	QMI8658 6 assi, su stesso bus I2C, indirizzo 0x6B
Storage	microSD (TF) — usa schede ≤ 64 GB formattate FAT32

3. Mappa pin completa

Tabella verificata sulla documentazione WaveShare. È la mappa più importante del documento: ti dice cosa è occupato e, soprattutto, dove ci sono **condivisioni** tra periferiche.

Funzione	Segnale	GPIO	Note
Display AMOLED (QSPI)	CS	GPIO6	Chip select del pannello
Display AMOLED (QSPI)	PCLK / WRX_SCL	GPIO47	Clock - CONDIVISO con SD_CLK
Display AMOLED (QSPI)	D0 (RD/SDI)	GPIO18	Linea dati 0
Display AMOLED (QSPI)	D1 (DCX_RS)	GPIO7	Linea dati 1
Display AMOLED (QSPI)	D2 (DB0)	GPIO48	Linea dati 2
Display AMOLED (QSPI)	D3 (DB1)	GPIO5	Linea dati 3
Display AMOLED (QSPI)	RST	GPIO17	Reset pannello
Display AMOLED (QSPI)	TE / SDO	GPIO9 / GPIO8	Tearing/data-out (non usati dallo sketch base)
Touch FT3168 + IMU (I2C)	SCL	GPIO39	Clock I2C - dedicato, non riusabile
Touch FT3168 + IMU (I2C)	SDA	GPIO40	Dati I2C - dedicato, non riusabile
Touch FT3168	INT	GPIO41	Interrupt touch
IMU QMI8658	INT1 / INT2	GPIO45 / GPIO46	Interrupt IMU
microSD (SPI)	CLK	GPIO47	CONDIVISO con il PCLK del display
microSD (SPI)	MISO	GPIO8	CONDIVISO con SDO del display
microSD (SPI)	MOSI	GPIO42	Dedicato SD
microSD (SPI)	CS	GPIO9	Dedicato SD (= TE display)
Alimentazione	BAT_ADC	GPIO1	Lettura tensione batteria
USB	D- / D+	GPIO19 / GPIO20	USB nativo
UART0	TX / RX	GPIO43 / GPIO44	Seriale di debug

3.1 GPIO liberi per le tue periferiche

Per un LED, un relè, un pulsante esterno, ecc., usa GPIO che non compaiono sopra. Scelte **sicure**: GPIO 2, 3, 4, 10, 11, 12, 13, 14, 15, 16, 21.

Da evitare come uscite generiche: GPIO0 (strap BOOT), GPIO43/44 (seriale USB-UART di debug), e in particolare **GPIO 26 e 33-37**: su questo modulo con PSRAM octal sono usati internamente dalla PSRAM/Flash anche se fisicamente portati su header. Non usarli come I/O.

3.2 Condivisioni da ricordare

- **I2C unico** (GPIO39 SCL / GPIO40 SDA) per touch e IMU: stesso bus, indirizzi diversi. Questi due pin sono dedicati e non riutilizzabili per altro.

- **SD ↔ display**: la SD condivide clock (GPIO47) e una linea dati (GPIO8) col pannello. Coesistono ma vanno gestite con attenzione (sezione 13).

4. Setup di Arduino IDE

Installa il core ESP32 di Espressif (gestore schede, URL [espressif.github.io/arduino-esp32/package_index.json](https://github.com/espressif/arduino-esp32/package_index.json)). Poi in **Tools**:

Board	ESP32S3 Dev Module
Flash Size	16MB (128Mb)
Partition Scheme	16M Flash (3MB APP/9.9MB FATFS)
PSRAM	OPI PSRAM
USB CDC On Boot	Enabled
CPU Frequency	240 MHz

Librerie: LVGL (Waveshare valida ufficialmente la **8.3.11**; la 8.4.x funziona allo stesso modo per tutto ciò che usiamo) e `esp_lcd_sh8601` di Waveshare. La libreria LVGL richiede un file `lv_conf.h`.

4.1 lv_conf.h — le voci che contano

```
#define LV_COLOR_DEPTH      16      // RGB565, obbligatorio per SH8601
#define LV_COLOR_16_SWAP    1      // se i colori sono invertiti, prova a cambiarlo
#define LV_TICK_CUSTOM      0      // usiamo lv_tick_inc() via esp_timer
#define LV_MEM_SIZE         (48 * 1024U) // alza se gli oggetti non vengono creati
#define LV_USE_PERF_MONITOR  0      // metti 1 per vedere FPS/CPU a schermo (debug)
```

Se i colori appaiono invertiti (rosso↔blu o "negativo"), il primo parametro da provare è `LV_COLOR_16_SWAP`. Su Arduino-ESP32 il tick FreeRTOS è già a 1000 Hz, quindi non devi cambiarlo (in ESP-IDF invece sì).

4.2 Tenere lv_conf.h a livello di progetto

Di default `lv_conf.h` va nella cartella `libraries/` (accanto a `lvgl`) ed è **globale**: una sola configurazione per tutti gli sketch, e fuori dal repository del progetto. Per averlo invece **dentro la cartella dello sketch** (così entra in git e ogni progetto ha le sue impostazioni) si sfrutta la macro `LV_CONF_INCLUDE_SIMPLE`: quando è definita, LVGL include la config con un semplice `#include "lv_conf.h"`, risolto sul percorso di include — e la cartella dello sketch è automaticamente nel percorso di include su Arduino.

1. Copia `lv_conf.h` nella cartella dello sketch (con il primo `#if 0` già messo a `#if 1`).
2. Aggiungi un file `build_opt.h` nella stessa cartella (nuova tab in Arduino IDE) con una sola riga:

```
-DLV_CONF_INCLUDE_SIMPLE -DLV_LVGL_H_INCLUDE_SIMPLE
```

3. Rimuovi o rinomina il `lv_conf.h` globale in `libraries/` per evitare ambiguità su quale venga preso.

4. **Chiudi e riapri l'IDE** (o fai un build pulito): la cache di Arduino conserva la vecchia config e senza riavvio le modifiche agli include possono non applicarsi.

Il punto cruciale. Non basta scrivere `#define LV_CONF_INCLUDE_SIMPLE` in cima al `.ino`: i sorgenti di LVGL (`lvgl/src/*.c`) sono unità di traduzione separate e non vedono quel define, generando comportamenti incoerenti o errori tipo `lvgl.h: No such file`. Il flag in `build_opt.h` invece è un define **globale** applicato a tutti i file compilati, LVGL compreso: per questo è l'unico modo affidabile su Arduino IDE. Il secondo define, `LV_LVGL_H_INCLUDE_SIMPLE`, serve ai file generati da SquareLine, che includono `lvgl.h` con la stessa logica.

In alternativa, per un nome o percorso personalizzato, nello stesso `build_opt.h` puoi usare un percorso assoluto (meno portabile, mette path nel repo):

```
-DLV_CONF_PATH="C:/Users/.../mio_progetto/lv_conf.h"
```

Tutto questo non interferisce con i re-export di SquareLine: `lv_conf.h` è indipendente dai file `ui_*`.

5. Struttura del progetto

Tutti i file vanno nella stessa cartella dello sketch. La regola è: **boilerplate isolato**, sketch con solo la tua logica.

<code>WSOLED.ino</code>	il tuo programma: setup/loop, WiFi, SD, sensori
<code>lvgl_port.h / .cpp</code>	display + touch + LVGL + task + mutex + comandi (non lo tocchi)
<code>esp_lcd_sh8601.h / .c</code>	driver pannello SH8601 (Espressif/Waveshare)
<code>touch_bsp.h / .c</code>	driver touch FT3168 (fornito Waveshare)
<code>ui.h, ui.c, ui_events.c, ...</code>	export di SquareLine Studio
<code>lv_conf.h</code>	config LVGL a livello di progetto (vedi 4.2)
<code>build_opt.h</code>	flag di compilazione (-DLV_CONF_INCLUDE_SIMPLE ...)

Il nome del `.ino` DEVE coincidere col nome della cartella (vincolo Arduino). Copiando il template in `MioProgetto/`, rinomina lo sketch in `MioProgetto.ino`.

6. Architettura: il modello a task + mutex

LVGL **non è thread-safe**. Tutto il rendering (ridisegno, animazioni, gestione eventi) avviene dentro una sola funzione, `lv_timer_handler()`, che noi facciamo girare in un **task FreeRTOS dedicato**. Quando un *altro* contesto (il `loop()`, un task sensori, una callback WiFi) vuole leggere o modificare un oggetto LVGL, deve prima prendere un **mutex** condiviso, altrimenti due esecuzioni concorrenti corrompono le strutture interne di LVGL.

La regola d'oro. Tocchi oggetti LVGL da fuori dal task LVGL? Allora avvolgi sempre il codice in `lvgl_lock(-1) ... lvgl_unlock()`. Sei *dentro* una callback di evento LVGL? Allora NON prendere il lock (ce l'hai già): lo faresti andare in deadlock.

Perché ti serve davvero questo modello: appena aggiungi WiFi, SD e sensori, aggiornerai la UI da contesti diversi dal task LVGL. Senza mutex i sintomi sono crash casuali, glitch grafici e blocchi difficili da diagnosticare.

7. Il modulo `lvgl_port`

L'header espone solo le funzioni pubbliche. Tutto il resto è incapsulato nel `.cpp`.

```
#ifndef LVGL_PORT_H
#define LVGL_PORT_H
#include <stdbool.h>
#include <stdint.h>
#include <stddef.h>
#ifdef __cplusplus
extern "C" {
#endif

void lvgl_port_init(void);           // init display+touch+LVGL+task. Una volta in setup().
bool lvgl_lock(int timeout_ms);     // -1 = attende all'infinito. Prima di toccare LVGL.
void lvgl_unlock(void);             // rilascia il lock.
void lcd_command(uint8_t cmd, const uint8_t *data, size_t len); // comando QSPI
void lcd_set_brightness(uint8_t level); // 0x51, 0..255

#ifdef __cplusplus
}
#endif
#endif
```

La sequenza di init del pannello SH8601 (dentro al `.cpp`) è tarata esattamente sul 536×240. Gli indirizzi `0x0217` = 535 colonne e `0xEF` = 239 righe non vanno cambiati; `0x36` controlla l'orientamento, `0x53` abilita il controllo luminosità e `0x51` imposta la luminosità.

```
static const sh8601_lcd_init_cmd_t lcd_init_cmds[] = {
    {0x11, (uint8_t[]){0x00}, 0, 120}, // Sleep Out
    {0x36, (uint8_t[]){0xF0}, 1, 0},   // MADCTL (landscape)
    {0x3A, (uint8_t[]){0x55}, 1, 0},   // Pixel format RGB565
    {0x53, (uint8_t[]){0x20}, 1, 0},   // WRCTRLD: BCTRL=1 (abilita 0x51)
    {0x2A, (uint8_t[]){0x00, 0x00, 0x02, 0x17}, 4, 0}, // Colonne 0..535
    {0x2B, (uint8_t[]){0x00, 0x00, 0x00, 0xEF}, 4, 0}, // Righe 0..239
    {0x29, (uint8_t[]){0x00}, 0, 10},  // Display On
    {0x51, (uint8_t[]){0xFF}, 1, 0},    // Luminosita' massima
};
```

Il mutex pubblico e il task di rendering sono il cuore del modello a thread:

```
bool lvgl_lock(int timeout_ms) {
    const TickType_t ticks = (timeout_ms == -1) ? portMAX_DELAY
                                                : pdMS_TO_TICKS(timeout_ms);
    return xSemaphoreTake(lvgl_mux, ticks) == pdTRUE;
}
void lvgl_unlock(void) { xSemaphoreGive(lvgl_mux); }
```

```
static void lvgl_task(void *arg) {
    uint32_t delay_ms = LVGL_TASK_MAX_DELAY_MS;
    while (1) {
        if (lvgl_lock(-1)) {                // il task tiene il lock...
            delay_ms = lv_timer_handler();  // ...mentre LVGL lavora
            lvgl_unlock();
        }
        if (delay_ms > LVGL_TASK_MAX_DELAY_MS) delay_ms = LVGL_TASK_MAX_DELAY_MS;
        else if (delay_ms < LVGL_TASK_MIN_DELAY_MS) delay_ms = LVGL_TASK_MIN_DELAY_MS;
        vTaskDelay(pdMS_TO_TICKS(delay_ms));
    }
}
```

Il tick di LVGL viene incrementato da un timer hardware (`esp_timer`) che chiama `lv_tick_inc()`: questa è l'unica funzione LVGL che si può chiamare fuori dal lock in sicurezza.

8. Lo sketch principale

Ordine in `setup()`: prima il modulo, poi la UI (sotto lock, perché crea oggetti), poi le tue periferiche.

```
void setup() {
    Serial.begin(115200);

    lvgl_port_init();                // 1) display + touch + LVGL + task rendering

    if (lvgl_lock(-1)) {             // 2) la UI crea oggetti LVGL -> serve il lock
        ui_init();
        lvgl_unlock();
    }

    // 3) le tue periferiche: WiFi, SD, sensori...
}
```

Aggiornare la UI periodicamente dal `loop()` — nota il lock:

```

void loop() {
    if (millis() - last_update >= 1000) {
        last_update = millis();
        float t = read_sensor();
        char buf[16];
        snprintf(buf, sizeof(buf), "%.1f C", t);

        if (lvgl_lock(-1)) { // SEMPRE sotto lock dal loop
            lv_label_set_text(ui_LabelTemp, buf); // <- tuo oggetto SquareLine
            lvgl_unlock();
        }
    }
    delay(5);
}

```

9. Workflow con SquareLine Studio

In SquareLine imposti risoluzione **536×240**, profondità colore **16 bit** e target **Arduino with LVGL** con versione LVGL **8.3.x** (deve combaciare con la libreria installata). Quando esporti i file UI ottieni `ui.h`, `ui.c` e i file degli asset/screen. Nel tuo sketch includi `ui.h` e chiami `ui_init()` una volta (sotto lock).

- Esporta **solo i file UI** nella cartella dello sketch: `ui.h/ui.c` sovrascrivono gli stub. **Non** usare il `.ino` né il driver TFT_eSPI di SquareLine.
- Il file `ui_events.c` (la tua logica eventi) **non viene sovrascritto** al re-export.
- Se rinomini oggetti in SquareLine, i nomi `ui_*` cambiano: aggiorna i riferimenti.

10. Gestire gli eventi LVGL

Ci sono due modi per agganciare un'azione a un widget. La firma del callback è sempre `void nome(lv_event_t * e)`.

10.1 Evento definito in SquareLine ("Call function")

Nel pannello *Events* del widget aggiungi un evento, scegli il trigger (`CLICKED`, `VALUE_CHANGED`, ...) e l'azione *Call function* con un nome. All'export trovi lo stub vuoto in `ui_events.c`: scrivi lì il corpo.

```

// in ui_events.c (lo stub e' generato da SquareLine, scrivi qui il corpo)
#include "ui.h"

void on_button_clicked(lv_event_t * e) // nome scelto in SquareLine
{
    // Siamo GIA' dentro il task LVGL, mutex gia' preso: niente lvgl_lock() qui!
    lv_label_set_text(ui_LabelStatus, "Premuto!");
}

```

10.2 Evento agganciato a mano

Per trigger o oggetti che non hai configurato in SquareLine, usa `lv_obj_add_event_cb()` dopo `ui_init()`. La **registrazione** avviene dal `setup()`, fuori dal task, quindi sotto lock.


```
static void slider_cb(lv_event_t * e) {
    lv_obj_t * s = lv_event_get_target(e);
    int32_t v = lv_slider_get_value(s);
    // usa v...
}

// in setup(), DOPO ui_init(): la REGISTRAZIONE avviene fuori dal task -> lock
if (lvgl_lock(-1)) {
    lv_obj_add_event_cb(ui_Slider1, slider_cb, LV_EVENT_VALUE_CHANGED, NULL);
    lvgl_unlock();
}
```

Dentro un callback di evento (entrambi i casi): sei già nel task LVGL col mutex preso, quindi (1) tocchi gli oggetti LVGL liberamente **senza** lock; (2) **non** chiamare `lvgl_lock()` → deadlock; (3) tieni il callback **corto e non bloccante**. Per leggere il contesto: `lv_event_get_target(e)`, `lv_event_get_code(e)`, `lv_event_get_user_data(e)`.

11. Comandare un GPIO / LED da un evento

Comandare un GPIO è **istantaneo e non bloccante** e non tocca LVGL: lo fai **direttamente nel callback**, senza lock. L'unico vincolo pratico è che `ui_events.c` è un file **C**, quindi lì non puoi includere `Arduino.h`.

11.1 Strada A — API GPIO di ESP-IDF (consigliata, C puro)

```
// in ui_events.c (file C): API GPIO di ESP-IDF, niente Arduino.h
#include "ui.h"
#include "driver/gpio.h"

#define LED_PIN  GPIO_NUM_15          // un GPIO LIBERO (vedi tabella pin)

void on_led_button_clicked(lv_event_t * e) {
    static bool on = false;
    on = !on;
    gpio_set_level(LED_PIN, on);      // istantaneo: inline nel callback, no lock
}
```

```
// in setup() (file .ino, C++): configura il pin una volta
pinMode(15, OUTPUT);
digitalWrite(15, LOW);
```

11.2 Strada B — funzione glue se preferisci l'API Arduino

```
// nel .ino (C++): funzione glue con linkage C, visibile da ui_events.c
extern "C" void app_toggle_led(void) {
    digitalWrite(15, !digitalRead(15));
}
```

```
// in ui_events.c
void app_toggle_led(void); // dichiarazione
void on_led_button_clicked(lv_event_t * e) { app_toggle_led(); }
```

In alternativa registra l'evento tu nel `.cpp` (sotto lock, vedi 10.2): lì il callback è già C++ e usi `digitalWrite()` direttamente, senza glue.

12. Operazioni lente o bloccanti dagli eventi

Se al click devi fare qualcosa di **lento o bloccante** (scrivere su SD, una richiesta HTTP), NON farlo nel callback: bloccheresti il rendering. Pattern corretto: nel callback setti solo un flag, il lavoro vero lo fai nel `loop()` o in un task, fuori dal callback.

```
volatile bool g_do_save = false;

void on_save_clicked(lv_event_t * e) { g_do_save = true; } // callback CORTO

// nel loop() (o in un tuo task):
if (g_do_save) {
    g_do_save = false;
    write_log_to_sd(); // lavoro lento: FUORI dal lock LVGL
    if (lvgl_lock(-1)) { // aggiorni la UI col risultato? -> lock
        lv_label_set_text(ui_LabelStatus, "Salvato");
        lvgl_unlock();
    }
}
```

Discriminante: veloce/non bloccante (GPIO, set di una variabile) → inline nel callback. Lento/bloccante (SD, rete, sensori lenti) → deferito al loop/task.

13. Integrare le periferiche

13.1 WiFi

Lo stack WiFi vuole RAM interna: se vedi allocazioni fallite, riduci i buffer LVGL (sezione 15). Aggiorna la UI sullo stato WiFi sempre sotto lock.

```
#include <WiFi.h>

// in setup(), dopo ui_init():
WiFi.begin("SSID", "password");

// in loop(), aggiorna la UI quando lo stato cambia:
static wl_status_t last = WL_IDLE_STATUS;
wl_status_t now = WiFi.status();
if (now != last) {
    last = now;
    const char* txt = (now == WL_CONNECTED) ? "WiFi OK" : "WiFi...";
    if (lvgl_lock(-1)) {
        lv_label_set_text(ui_LabelWifi, txt);
        lvgl_unlock();
    }
}
```

13.2 microSD — attenzione ai pin condivisi

Come visto nella tabella pin, la SD condivide **clock (GPIO47)** e **una linea dati (GPIO8)** con il display. Non puoi quindi assegnarle un bus SPI indipendente con pin liberi: usa i pin reali (**CLK=47**, **MISO=8**, **MOSI=42**, **CS=9**) e assicurati che accessi a SD e display non avvengano in parallelo. Il riferimento autorevole per questa scheda è il demo Arduino **SD_Test** di Waveshare. Regole pratiche: scheda ≤ 64 GB, formattata FAT32.

13.3 IMU QMI8658 e altri device I2C

L'IMU sta sullo stesso bus I2C del touch (SDA=40, SCL=39). Se vuoi leggerlo, riusa quel bus (**I2C_NUM_0** /stessi pin) e dialoga con l'indirizzo **0x6B** (touch su **0x38**). Non aprire un secondo bus sugli stessi pin.

```
#include <Wire.h>
#define IIC_SDA 40
#define IIC_SCL 39
// Wire.begin(IIC_SDA, IIC_SCL); // stesso bus del touch (indirizzi 0x38 touch, 0x6B IMU)
```

13.4 Aggiornare la UI da un tuo task

```
static void sensor_task(void *arg) {
    for (;;) {
        float t = read_sensor();
        char buf[16];
        snprintf(buf, sizeof(buf), "%.1f C", t);
        if (lvgl_lock(-1)) { // task diverso da LVGL -> lock
            lv_label_set_text(ui_LabelTemp, buf);
            lvgl_unlock();
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
// in setup(): xTaskCreate(sensor_task, "sensor", 4096, NULL, 1, NULL);
```

14. Inviare comandi al pannello (QSPI)

Molti riferimenti comandi in giro (incluso il foglio RM67162) sono scritti per la libreria **Arduino_GFX** (`bus->writeC8D8(...)`). Questo progetto usa invece il path **esp_lcd**: i **codici** dei comandi sono identici (SH8601 e RM67162 condividono il set), cambia solo il **trasporto**.

Il framing QSPI — ovvero `[0x02][0x00][CMD][0x00][DATA...]` — lo costruisce `esp_lcd`: la parola di comando a 32 bit è `(0x02<<24) | (cmd<<8)` e l'IO è configurato con `lcd_cmd_bits = 32`. Hai due modi per mandare un comando.

14.1 Una tantum all'avvio: aggiungilo a `lcd_init_cmds[]`

Il modo più semplice: il loop di init lo invia col framing corretto. Esempi (vedi sezione 7): `{0x53, {0x20}, 1, 0}` per abilitare la luminosità, `{0xB0, {0x04}, 1, 0}` per l'High Brightness Mode.

14.2 A runtime: l'helper `lcd_command()`

Nel `lvgl_port` l'handle del bus viene salvato in una variabile di file e l'header espone due funzioni: `lcd_command()` e `lcd_set_brightness()`.

```
// in lvgl_port.cpp
#define LCD_QSPI_WRITE_CMD (0x02UL)
static esp_lcd_panel_io_handle_t s_io_handle = NULL; // salvato in lvgl_port_init()

void lcd_command(uint8_t cmd, const uint8_t *data, size_t len) {
    // Framing QSPI SH8601/RM67162: [0x02][0x00][CMD][0x00][DATA...]
    uint32_t qspi_cmd = (LCD_QSPI_WRITE_CMD << 24) | ((uint32_t)cmd << 8);
    esp_lcd_panel_io_tx_param(s_io_handle, qspi_cmd, data, len);
}

void lcd_set_brightness(uint8_t level) {
    lcd_command(0x51, &level, 1);
}
```

```
// dentro lvgl_port_init(), subito dopo esp_lcd_new_panel_io_spi(...):
s_io_handle = io_handle;
```

Uso dallo sketch (a runtime sempre sotto lock, è lo stesso bus del rendering):

```
// a runtime (dal loop o da un task): SEMPRE sotto lock, e' lo stesso bus del rendering
if (lvgl_lock(-1)) {
    lcd_set_brightness(180); // ~70%
    lcd_command(0x39, NULL, 0); // idle mode on (8 colori, risparmio)
    lcd_command(0xB0, (uint8_t[]){0x04}, 1); // HBM on (high brightness)
    lvgl_unlock();
}
```

Da non fare a runtime: cambiare `0x36` (MADCTL, orientamento) o `0x3A` (COLMOD, formato pixel) mentre LVGL disegna → glitch dei frame. Questi vanno solo nell'init. Sono invece sicuri a runtime: luminosità (0x51), idle mode (0x39/0x38), HBM (0xB0), inversione (0x21/0x20), all-pixels on/off (0x23/0x22).

14.3 Comandi senza parametro

Si inviano con `lcd_command(cmd, NULL, 0)`.

Hex	Comando	Descrizione
0x01	SWRESET	Software reset
0x10	SLPIN	Sleep in (consumo minimo)
0x11	SLPOUT	Sleep out (riattiva)
0x13	NORON	Normal display mode
0x20	INVOFF	Inversione colori off
0x21	INVON	Inversione colori on
0x22	ALLPOFF	Tutti i pixel spenti (nero forzato)
0x23	ALLPON	Tutti i pixel accesi (bianco forzato)
0x28	DISPOFF	Display off
0x29	DISPON	Display on
0x38	IDMOFF	Idle mode off (colori pieni)
0x39	IDMON	Idle mode on (8 colori, risparmio)

14.4 Comandi con un byte di parametro

Si inviano con `lcd_command(cmd, &valore, 1)`.

Hex	Comando	Valori	Descrizione
0x51	WRDISBV	0x00..0xFF	Luminosita' (usa <code>lcd_set_brightness</code>)
0x53	WRCTRLD	0x20 / 0x2C	0x20=abilita 0x51; 0x2C=+dimming hw
0x36	MADCTL	vedi sez.15	Orientamento (attuale 0xF0)
0x3A	COLMOD	0x55/0x66/0x77	Formato pixel (0x55=RGB565)
0xB0	SetHBMMode	0x04 / 0x00	High Brightness Mode on/off
0x58	IMGEHCCTR	0x04/0x05/0x07	Sunlight Readable on (liv. 0/1/2)
0x35	TEON	0x00 / 0x01	Tearing effect line on

Dettagli utili. **WRCTRLD** (0x53): il bit 5 (BCTRL) deve essere 1 perché 0x51 abbia effetto — **0x20** = controllo manuale, **0x2C** = manuale + dimming hardware. **SetHBMMode** (0xB0): **0x04** accende la modalità alta luminosità (oltre 0xFF), utile all'aperto. **IMGEHCCTR** (0x58): **0x04/0x05/0x07** attivano il Sunlight Readable Enhancement a livello crescente.

14.5 Leggere i registri

La lettura usa l'opcode **0x03** e `esp_lcd_panel_io_rx_param()`. L'helper `lcd_read_register()` riempie un buffer con i byte richiesti.

```
// in lvgl_port.cpp
#define LCD_QSPI_READ_CMD    (0x03UL)

bool lcd_read_register(uint8_t cmd, uint8_t *data, size_t len) {
    uint32_t qspi_cmd = (LCD_QSPI_READ_CMD << 24) | ((uint32_t)cmd << 8);
    return esp_lcd_panel_io_rx_param(s_io_handle, qspi_cmd, data, len) == ESP_OK;
}
```

```
// esempio: rileggere la luminosita' impostata (RDISBV = 0x52)
uint8_t bv = 0;
if (lvgl_lock(-1)) {
    if (lcd_read_register(0x52, &bv, 1))
        Serial.printf("Luminosita' attuale: %u\n", bv);
    lvgl_unlock();
}
```

Affidabilità. Sui pannelli AMOLED QSPI la **lettura** dei registri è molto meno robusta della scrittura: a seconda della board e del cablaggio della linea dati può restituire valori non validi o zero. Consideralo uno strumento di **diagnostica**, non un meccanismo su cui basare la logica applicativa. Vale comunque la stessa regola di lock degli altri comandi.

Hex	Comando	Byte	Descrizione
0x04	RDDID	3	Display ID
0x0A	RDDPM	1	Power mode
0x0B	RDDMADCTR	1	MADCTL attuale (orientamento)
0x0C	RDDCOLMOD	1	Formato pixel attuale
0x0F	RDDSDR	1	Self-diagnostic result
0x52	RDDISBV	1	Luminosita' attuale
0x54	RDCTRLD	1	CTRL display (BCTRL/DD)

15. Rotazione, luminosità e prestazioni

Rotazione: registro `0x36` (MADCTL) nella sequenza di init. Il valore attuale `0xF0` dà il landscape; cambiandolo vanno aggiornati anche `hor_res/ver_res` del driver LVGL e la mappatura del touch in `touch_bsp.c` (scambio/mirror degli assi).

Luminosità a runtime: `lcd_set_brightness(0..255)` (sezione 14).

Tuning memoria: i due buffer LVGL usano $LCD_V_RES/4 = 60$ righe (≈ 128 KB di RAM DMA interna). Scendi a `/8` per liberare RAM (utile con WiFi), sali a `/2` per più fluidità. In sviluppo, `LV_USE_PERF_MONITOR 1` mostra FPS e carico CPU a schermo.

16. Errori comuni e come uscirne

Sintomo	Causa probabile / rimedio
Tutto si blocca quando aggiorni la UI da un evento	Hai chiamato <code>lvgl_lock()</code> dentro un callback: rimuovilo, nel callback il lock c'è già.
Crash casuali con WiFi/sensori attivi	Stai toccando oggetti LVGL da un altro task senza lock: avvolgi in <code>lvgl_lock/unlock</code> .
Schermo bianco/nero o niente UI	Hai chiamato <code>ui_init()</code> fuori dal lock o prima di <code>lvgl_port_init()</code> ; verifica l'ordine in <code>setup()</code> .
Colori invertiti	Cambia <code>LV_COLOR_16_SWAP</code> in <code>lv_conf.h</code> .
Luminosità (0x51) ignorata	Manca l'abilitazione BCTRL: aggiungi <code>{0x53,{0x20},1,0}</code> all'init.
Touch sfasato/specchiato dopo rotazione	Aggiorna <code>swap_xy</code> / <code>mirror_x</code> / <code>mirror_y</code> coerentemente col valore di 0x36.
Allocazioni fallite all'avvio del WiFi	RAM interna esaurita: riduci i buffer LVGL a <code>V_RES/8</code> .
La SD non monta	Scheda > 64 GB o non FAT32; oppure conflitto col display: segui il demo <code>SD_Test</code> .
<code>lvgl.h: No such file</code> dai file <code>ui_*</code>	Manca <code>-DLV_LVGL_H_INCLUDE_SIMPLE</code> in <code>build_opt.h</code> .
Upload non parte	Tieni premuto BOOT, premi e rilascia RST, rilascia BOOT, poi carica.

17. Cheat sheet

<code>lvgl_port_init()</code>	una volta in <code>setup()</code> , prima di <code>ui_init()</code>
<code>lvgl_lock(-1)</code> / <code>lvgl_unlock()</code>	attorno a ogni accesso LVGL fuori dal task
<code>lcd_command(cmd, data, len)</code>	comando al pannello (sotto lock a runtime)
<code>lcd_set_brightness(0..255)</code>	luminosità display
<code>lcd_read_register(cmd, buf, len)</code>	lettura registro (diagnostica)
<code>lv_label_set_text(obj, "...")</code>	testo di una label
<code>lv_obj_add/clear_state(obj, LV_STATE_*)</code>	stato (checked, disabled, ...)
<code>lv_obj_add/remove_flag(obj, LV_OBJ_FLAG_HIDDEN)</code>	mostra/nascondi un oggetto
<code>lv_obj_add_event_cb(obj, cb, code, data)</code>	aggancia evento (sotto lock)
<code>lv_event_get_target</code> / <code>_code</code> / <code>_user_data(e)</code>	contesto dentro un callback
<code>lv_scr_load(ui_ScreenX)</code>	cambia schermata

18. Risorse

- Wiki Waveshare ESP32-S3-AMOLED-1.91 (pinout, demo Arduino: SD_Test, WIFI_Test, IMU_Test, schematico).
- Documentazione ufficiale LVGL v8 (widget, eventi, stili, layout).
- SquareLine Studio — documentazione su export Arduino ed eventi.
- Datasheet SH8601 / RM67162 per il set comandi completo del controller.

Guida generata come riferimento per il tuo progetto. I dettagli hardware (pinout, condivisioni SD/I2C, indirizzi I2C, requisiti SD) provengono dalla documentazione Waveshare della scheda; i codici comandi dal foglio di riferimento RM67162/SH8601. Verifica sempre lo schematico ufficiale del tuo specifico revision di board prima di cablare periferiche esterne.